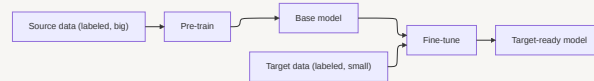


Transfer Learning — Complete Notes

Fine-Tuning

Setting: large labeled source dataset + small labeled target dataset. Pre-train on source → adapt to target.

Goal: reuse learned features while avoiding catastrophic forgetting.



Conservative training (avoid forgetting)

Two common constraints:

- **Output constraint:** keep outputs on source validation close to the original model
- **Parameter constraint:** keep weights close to source initialization (L2 / weight regularization)

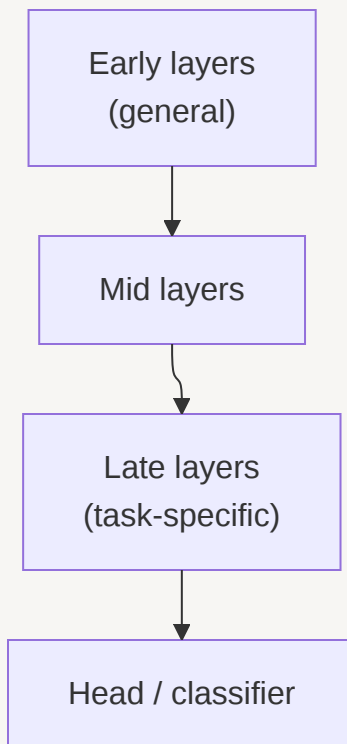
Layer transfer — copy and freeze (what to copy)

General rule: earlier layers = more general, later layers = more task-specific.

Modality	Usually transferable	Usually needs adaptation
Images (CNNs)	Early layers (edges/textures)	Late layers (task-specific)
Speech (ASR)	Often later layers (linguistic/phoneme)	Earlier layers (speaker/channel)

Two strategies after copying:

1. **Freeze copied layers, train the rest** (best when target data is tiny)
2. **Fine-tune all layers** (better when target data is enough)

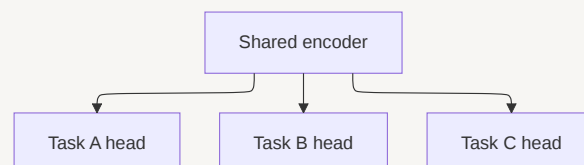


Yosinski et al. (NIPS 2014) — key takeaways

- Fine-tuning after transfer beats random init, even for deep transfers
- Freezing deep layers hurts more (co-adaptation issues)
- Transferability decreases with depth (higher layers more task-specific)

Multitask Learning

Idea: train one model on multiple tasks jointly. Shared representation learns general features; task heads specialize.



Multilingual speech recognition (example)

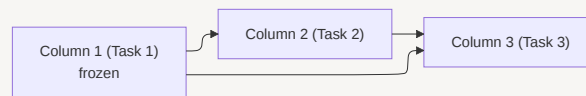
- Shared acoustic layers + separate output heads per language.
- Helps low-resource languages by sharing acoustic features.

Progressive Neural Networks (Rusu et al., 2016)

Idea: add a new “column” per task; old columns are frozen; lateral connections reuse previous features.

Pros: prevents catastrophic forgetting

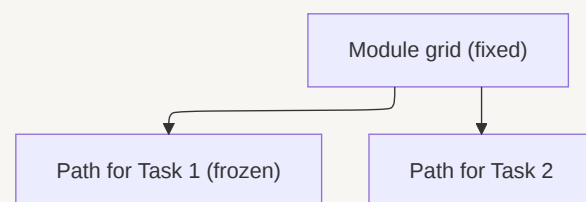
Cons: model size grows with #tasks (not scalable to many tasks)



PathNet (Fernando et al., 2017)

Idea: fixed-size network of modules; for each task, find a good path (subset of modules). Freeze that path; next task finds another path, possibly reusing some modules.

Why it matters: avoids Progressive NN growth while still reusing knowledge.

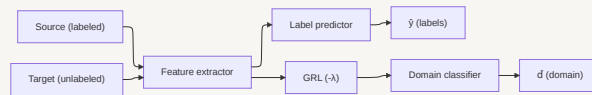


Domain-Adversarial Training (Ganin & Lempitsky, 2015)

Setting: source labeled, target unlabeled, distribution shift.

Goal: learn features that are:

- good for label prediction
- domain-invariant (source vs target indistinguishable)



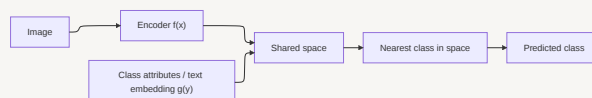
Key trick: Gradient Reversal Layer (GRL)

- Forward: identity
- Backward: multiplies gradient by $-\lambda$
- Effect: features become harder for the domain classifier to separate

Zero-Shot Learning

Problem: test classes never seen during training → softmax can't directly predict them.

Core idea: predict into an attribute/embedding space, then match to class descriptions.



Method 1 — Attribute vectors (quick)

- Train: image → attribute vector (using seen classes)
- Test: nearest attribute vector → class label
- Needs good attribute design

Method 2 — Shared embedding space (ranking loss idea)

- Map image features $f(x)$ and class descriptors $g(y)$ into same space

- Train to make correct class closer than incorrect ones (margin ranking)

Method 3 — Word embeddings (no manual attributes)

- Use Word2Vec/GloVe for class names as descriptors
 - Works when names capture semantics (not always)
-

Self-Taught Learning

Setting: large unlabeled source data from a different distribution + tiny labeled target data.

Pipeline:

1. Learn features from unlabeled source (e.g., autoencoder)
2. Re-represent target data using learned features
3. Train classifier on re-represented target

